You're also using a well-tested and mature JSON library to parse the input, which eliminates a lot of problems that can occur. You should always use well-established formats and libraries for processing all input to your API where possible. JSON is much better than the complex XML formats it replaced, but there are still often significant differences in how different libraries parse the same JSON.

> **LEARN MORE** Input parsing is a very common source of security vulnerabilities, and many widely used input formats are poorly specified resulting in differences in how they are parsed by different libraries. The ***LANGSEC*** movement (http://langsec.org) argues for the use of simple and unambiguous input formats and automatically generated parsers to avoid these issues.

Insecure deserialization

Although Java is a memory-safe language and so less prone to buffer overflow attacks, that does not mean it is immune from RCE attacks. Some *serialization* libraries that convert arbitrary Java objects to and from string or binary formats have turned out to be vulnerable to RCE attacks, known as an *insecure deserialization vulnerability* in the OWASP Top 10. This affects Java's built-in `Serializable` framework, but also parsers for supposedly safe formats like JSON have been vulnerable, such as the popular *Jackson Databind*[1]. The problem occurs because Java will execute code within the default constructor of any object being deserialized by these frameworks. Some classes included with popular Java libraries perform dangerous operations in their constructors, including reading and writing files and performing other actions. Some classes can even be used to load and execute attacker-supplied bytecode directly. Attackers can exploit this behavior by sending a carefully-crafted message that causes the vulnerable class to be loaded and executed.

The solution to these problems is to *whitelist* a known set of safe classes and refuse to deserialize any other class. Avoid frameworks that do not allow you to control which classes are deserialized. Consult the OWASP Deserialization Cheat Sheet for advice on avoid insecure deserialization vulnerabilities in several programming languages: https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html. You should take extra care when using a complex input format such as XML, as there are several specific attacks against such formats. OWASP maintains cheat sheets for secure